



Django

Let's create a Django-based movie website step by step, focusing on backend logic, authentication, forms, and plain HTML templates. I'll explain each step clearly, provide full code for each part, and include terminal commands where needed. We'll build the project from scratch, ensuring it's beginner-friendly and covers all requested features.

Step 1: Set Up the Django Project and App

Explanation:

We'll create a new Django project called `movie_website` and an app called `movies`. The project will hold the overall configuration, while the app will contain the models, views, and templates for the movie functionality. We'll also set up the templates folder and configure the project settings.

Terminal Commands:

```
# Install Django if not already installed
pip install django

# Create a new Django project
django-admin startproject movie_website

# Navigate to the project directory
cd movie_website

# Create a new app called 'movies'
python manage.py startapp movies

# Create a templates directory for HTML files
mkdir templates
```

Code:

1. Add the `movies` app and templates directory to `settings.py`

Update

`movie_website/settings.py` to include the `movies` app and configure the templates directory.

```
"""
Django settings for movie_website project.

Generated by 'django-admin startproject' using Django 4.2.

For more information on this file, see
<https://docs.djangoproject.com/en/4.2/topics/settings/>

For the full list of settings and their values, see
<https://docs.djangoproject.com/en/4.2/ref/settings/>
"""

from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

# Quick-start development settings - unsuitable for production
# See <https://docs.djangoproject.com/en/4.2/howto/deployment/checklist/>
```

```

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-@your-secret-key-here' # Replace with a secure key in production

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = []

# Application definition
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'movies', # Add the movies app
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'movie_website.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'], # Add templates directory
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    ],
]

WSGI_APPLICATION = 'movie_website.wsgi.application'

# Database
# <https://docs.djangoproject.com/en/4.2/ref/settings/#databases>
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

# Password validation

```

```
# <https://docs.djangoproject.com/en/4.2/ref/settings/#auth-password-validators>
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

# Internationalization
# <https://docs.djangoproject.com/en/4.2/topics/i18n/>
LANGUAGE_CODE = 'en-us'
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True

# Static files (CSS, JavaScript, Images)
# <https://docs.djangoproject.com/en/4.2/howto/static-files/>
STATIC_URL = 'static/'

# Default primary key field type
# <https://docs.djangoproject.com/en/4.2/ref/settings/#default-auto-field>
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

# Redirect to home page after login/logout
LOGIN_REDIRECT_URL = 'movie_list'
LOGOUT_REDIRECT_URL = 'movie_list'
```

Explanation of **settings.py**:

- Added `movies` to `INSTALLED_APPS` to register the app.
- Configured `TEMPLATES['DIRS']` to point to the `templates` folder in the project root.
- Set `LOGIN_REDIRECT_URL` and `LOGOUT_REDIRECT_URL` to redirect users to the movie list page after login/logout.
- The rest are default Django settings for a basic project using SQLite.

Step 2: Create Models and Admin Registration

Explanation:

We'll define two models: `Movie` and `Review`. The `Movie` model stores movie details, and the `Review` model links reviews to movies and users. We'll also register these models in the admin panel for admin management. After defining models, we'll create and apply migrations.

Code:

1. Define models in `movies/models.py`

The

`Movie` model includes fields for title, description, image, release date, and category. The `Review` model links to a movie and user, with fields for content and rating.

```
from django.db import models
from django.contrib.auth.models import User

class Movie(models.Model):
```

```

CATEGORY_CHOICES = [
    ('Action', 'Action'),
    ('Thriller', 'Thriller'),
    ('Comedy', 'Comedy'),
    ('Drama', 'Drama'),
    ('Sci-Fi', 'Sci-Fi'),
]

title = models.CharField(max_length=200)
description = models.TextField()
image = models.URLField(max_length=500) # Store image as URL for simplicity
release_date = models.DateField()
category = models.CharField(max_length=50, choices=CATEGORY_CHOICES)
created_at = models.DateTimeField(auto_now_add=True)

def __str__(self):
    return self.title

def average_rating(self):
    reviews = self.review_set.all()
    if reviews:
        return sum(review.rating for review in reviews) / reviews.count()
    return 0

class Review(models.Model):
    movie = models.ForeignKey(Movie, on_delete=models.CASCADE)
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    content = models.TextField()
    rating = models.IntegerField(choices=[(i, i) for i in range(1, 6)]) # 1 to 5
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = ['movie', 'user'] # One review per user per movie

    def __str__(self):
        return f"{self.user.username} - {self.movie.title}"

```

Explanation of models.py:

- **Movie** :
 - **title** : Movie name (max 200 characters).
 - **description** : Movie summary.
 - **image** : URL to an image (simplifies storage for this project).
 - **release_date** : Release date of the movie.
 - **category** : Choice field with predefined categories (Action, Thriller, etc.).
 - **average_rating** : Method to calculate the average rating from reviews.
- **Review** :
 - **movie** : Foreign key to **Movie** (deletes reviews if movie is deleted).
 - **user** : Foreign key to Django's **User** model.
 - **content** : Text of the review.
 - **rating** : Integer from 1 to 5.
 - **unique_together** : Ensures a user can review a movie only once.

1. Register models in **movies/admin.py**

This allows admins to manage movies and reviews via the Django admin panel.

```
from django.contrib import admin
from .models import Movie, Review

@admin.register(Movie)
class MovieAdmin(admin.ModelAdmin):
    list_display = ['title', 'category', 'release_date', 'average_rating']
    list_filter = ['category']
    search_fields = ['title']

@admin.register(Review)
class ReviewAdmin(admin.ModelAdmin):
    list_display = ['movie', 'user', 'rating', 'created_at']
    list_filter = ['movie', 'user']
    search_fields = ['movie__title', 'user__username']
```

Explanation of `admin.py`:

- `MovieAdmin` : Displays title, category, release date, and average rating in the admin list view. Allows filtering by category and searching by title.
- `ReviewAdmin` : Displays movie, user, rating, and creation date. Allows filtering by movie/user and searching by movie title or username.

Terminal Commands:

```
# Create migrations for the models
python manage.py makemigrations

# Apply migrations to create database tables
python manage.py migrate

# Create a superuser (admin) for the admin panel
python manage.py createsuperuser
# Follow prompts to set username, email, and password
```

Explanation of Commands:

- `makemigrations` : Generates migration files based on model changes.
- `migrate` : Applies migrations to create/update database tables.
- `createsuperuser` : Creates an admin user to access `/admin` and manage movies/reviews.

Step 3: Set Up Base Template and Navbar

Explanation:

We'll create a base HTML template with a navbar that appears on all pages. The navbar will show the website name and conditional links based on user authentication (Signup/Login for anonymous users, username/Logout for logged-in users). Other templates will extend this base template.

Code:

- Create base template in `templates/base.html`**
This defines the navbar and a block for content.

```
<!DOCTYPE html>
<html>
<head>
  <title>Movie Website</title>
</head>
<body>
  <nav>
    <h1>Movie Website</h1>
    {% if user.is_authenticated %}
```

```

        <p>Welcome, {{ user.username }} | <a href="{% url 'logout' %}">Logout</a></p>
    {% else %}
        <p><a href="{% url 'signup' %}">Signup</a> | <a href="{% url 'login' %}">Login</a></p>
    {% endif %}
</nav>
<div>
    {% if messages %}
        {% for message in messages %}
            <p>{{ message }}</p>
        {% endfor %}
    {% endif %}
    {% block content %}
    {% endblock %}
</div>
</body>
</html>

```

Explanation of base.html:

- The `<nav>` section includes the website title and conditional links:
 - If `user.is_authenticated`, show the username and a Logout link.
 - Otherwise, show Signup and Login links.
- Uses Django's `{% url %}` template tag to generate URLs dynamically.
- Displays messages from Django's messages framework (e.g., for success/error feedback).
- The `{% block content %}` allows child templates to inject their content.

Step 4: Set Up Authentication (Signup, Login, Logout)

Explanation:

We'll use Django's built-in authentication system to handle user signup, login, and logout. We'll create views, forms, and templates for these features, and set up URLs. The messages framework will provide feedback (e.g., "Successfully logged in").

Code:

1. Create forms in `movies/forms.py`

Define a form for user signup using Django's

`UserCreationForm`.

```

from django import forms
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User

class SignUpForm(UserCreationForm):
    email = forms.EmailField(required=True)

    class Meta:
        model = User
        fields = ['username', 'email', 'password1', 'password2']

```

Explanation of `forms.py`:

- `SignUpForm` extends `UserCreationForm` to include an email field.
- Fields include username, email, and two password fields (for confirmation).

1. Create views in `movies/views.py`

Define views for signup, login, and logout. We'll add more views later for other features.

```

from django.shortcuts import render, redirect
from django.contrib.auth import login, authenticate

```



```

from django.contrib.auth.forms import AuthenticationForm
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from .forms import SignUpForm

def signup(request):
    if request.method == 'POST':
        form = SignUpForm(request.POST)
        if form.is_valid():
            form.save()
            username = form.cleaned_data.get('username')
            messages.success(request, f'Account created for {username}!')
            return redirect('login')
    else:
        form = SignUpForm()
    return render(request, 'signup.html', {'form': form})

def user_login(request):
    if request.method == 'POST':
        form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            username = form.cleaned_data.get('username')
            password = form.cleaned_data.get('password')
            user = authenticate(username=username, password=password)
            if user is not None:
                login(request, user)
                messages.success(request, f'Welcome back, {username}!')
                return redirect('movie_list')
    else:
        form = AuthenticationForm()
    return render(request, 'login.html', {'form': form})

@login_required
def user_logout(request):
    messages.success(request, 'You have been logged out.')
    return redirect('logout')

```

Explanation of `views.py`:

- `signup` : Handles user registration. On POST, validates and saves the form, then redirects to login.
- `user_login` : Uses Django's `AuthenticationForm` for login. On valid POST, authenticates and logs in the user.
- `user_logout` : Uses `@login_required` to ensure only logged-in users can log out. Redirects to Django's built-in logout view.
- Uses `messages.success` to display feedback.

1. Create templates for signup and login

- **Signup template in `templates/signup.html` :**

```

{% extends 'base.html' %}
{% block content %}
<h2>Sign Up</h2>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Sign Up</button>
</form>
<a href="{% url 'login' %}">Already have an account? Login</a>
{% endblock %}

```


- **Login template in** `templates/login.html` :

```
{% extends 'base.html' %}
{% block content %}
<h2>Login</h2>
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Login</button>
</form>
<a href="{% url 'signup' %}">Don't have an account? Sign Up</a>
{% endblock %}
```

Explanation of templates:

- Both extend `base.html` to include the navbar.
- Use `{{ form.as_p }}` to render form fields as paragraphs.
- Include CSRF token for security.
- Provide links to switch between signup and login.

1. **Set up URLs in** `movie_website/urls.py`

Define project-level URLs, including auth URLs and the movies app.

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('movies.urls')), # Include movies app URLs
    path('accounts/', include('django.contrib.auth.urls')), # Built-in auth URLs
]
```

1. **Create app URLs in** `movies/urls.py`

Define URLs for the movies app.

```
from django.urls import path
from . import views

urlpatterns = [
    path('signup/', views.signup, name='signup'),
    path('login/', views.user_login, name='login'),
]
```

Explanation of URLs:

- `movie_website/urls.py` : Routes `/admin/` to the admin panel, `accounts/` to Django’s auth URLs (includes logout), and root (`"`) to the movies app.
- `movies/urls.py` : Defines URLs for signup and login views. More URLs will be added later.

Step 5: Implement Main Page (Movie List with Search and Filter)

Explanation:

The main page will display all movies, with a search box for title and a dropdown to filter by category. We’ll create a view to handle search/filter logic, a template for the movie list, and update URLs.

Code:

1. **Update** `movies/views.py`

Add a view for the movie list with search and filter functionality.


```

from django.shortcuts import render, redirect
from django.contrib.auth import login, authenticate
from django.contrib.auth.forms import AuthenticationForm
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from .forms import SignUpForm
from .models import Movie

def signup(request):
    if request.method == 'POST':
        form = SignUpForm(request.POST)
        if form.is_valid():
            form.save()
            username = form.cleaned_data.get('username')
            messages.success(request, f'Account created for {username}!')
            return redirect('login')
    else:
        form = SignUpForm()
    return render(request, 'signup.html', {'form': form})

def user_login(request):
    if request.method == 'POST':
        form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            username = form.cleaned_data.get('username')
            password = form.cleaned_data.get('password')
            user = authenticate(username=username, password=password)
            if user is not None:
                login(request, user)
                messages.success(request, f'Welcome back, {username}!')
                return redirect('movie_list')
    else:
        form = AuthenticationForm()
    return render(request, 'login.html', {'form': form})

@login_required
def user_logout(request):
    messages.success(request, 'You have been logged out.')
    return redirect('logout')

def movie_list(request):
    movies = Movie.objects.all()
    categories = Movie.CATEGORY_CHOICES
    search_query = request.GET.get('search', '')
    category = request.GET.get('category', '')

    if search_query:
        movies = movies.filter(title__icontains=search_query)
    if category:
        movies = movies.filter(category=category)

    return render(request, 'movie_list.html', {
        'movies': movies,
        'categories': categories,
        'search_query': search_query,
        'selected_category': category,
    })

```

Explanation of movie_list view:

- Fetches all movies by default.
- Gets `search` and `category` from GET parameters.
- Filters movies by title (case-insensitive) if `search_query` is provided.
- Filters by category if `category` is provided.
- Passes movies, categories, and query values to the template.

1. Create template in `templates/movie_list.html`

Displays the search/filter form and movie list.

```
{% extends 'base.html' %}
{% block content %}
<h2>Movies</h2>
<form method="get">
  <input type="text" name="search" value="{{ search_query }}" placeholder="Search by title">
  <select name="category">
    <option value="">All Categories</option>
    {% for cat in categories %}
      <option value="{{ cat.0 }}" {% if selected_category == cat.0 %}selected{% endif %}>{{ cat.1 }}</option>
    {% endfor %}
  </select>
  <button type="submit">Search</button>
</form>
<div>
  {% for movie in movies %}
    <div>
      <h3><a href="{% url 'movie_detail' movie.id %}">{{ movie.title }}</a></h3>
      
      <p>Rating: {{ movie.average_rating|floatformat:1 }}</p>
    </div>
  {% empty %}
    <p>No movies found.</p>
  {% endfor %}
</div>
{% endblock %}
```

Explanation of `movie_list.html`:

- Extends `base.html` for navbar.
- Form with `method="get"` for search and category filter:
 - Text input for title search, retaining `search_query`.
 - Dropdown for categories, with `selected` attribute for current category.
- Displays movies as cards with title (linked to detail page), image, and average rating.
- Shows “No movies found” if the queryset is empty.

1. Update `movies/urls.py`

Add URL for the movie list.

```
from django.urls import path
from . import views

urlpatterns = [
    path('signup/', views.signup, name='signup'),
    path('login/', views.user_login, name='login'),
    path('', views.movie_list, name='movie_list'),
]
```

Explanation: Added `movie_list` URL at the root path (`''`).

Step 6: Implement Movie Detail Page and Review Submission

Explanation:

The movie detail page will show movie details, all reviews, and a form for logged-in users to submit a review. We'll enforce that users can review a movie only once and redirect non-logged-in users to login.

Code:

1. Update `movies/forms.py`

Add a form for submitting reviews.

```
from django import forms
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User
from .models import Review

class SignUpForm(UserCreationForm):
    email = forms.EmailField(required=True)

    class Meta:
        model = User
        fields = ['username', 'email', 'password1', 'password2']

class ReviewForm(forms.ModelForm):
    class Meta:
        model = Review
        fields = ['content', 'rating']
```

Explanation: `ReviewForm` is a `ModelForm` for the `Review` model, including `content` and `rating` fields.

1. Update `movies/views.py`

Add a view for the movie detail page and review submission.

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth import login, authenticate
from django.contrib.auth.forms import AuthenticationForm
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from .forms import SignUpForm, ReviewForm
from .models import Movie, Review

def signup(request):
    if request.method == 'POST':
        form = SignUpForm(request.POST)
        if form.is_valid():
            form.save()
            username = form.cleaned_data.get('username')
            messages.success(request, f'Account created for {username}!')
            return redirect('login')
    else:
        form = SignUpForm()
    return render(request, 'signup.html', {'form': form})

def user_login(request):
    if request.method == 'POST':
        form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            username = form.cleaned_data.get('username')
            password = form.cleaned_data.get('password')
            user = authenticate(username=username, password=password)
            if user is not None:
```

```

        login(request, user)
        messages.success(request, f'Welcome back, {username}!')
        return redirect('movie_list')
    else:
        form = AuthenticationForm()
    return render(request, 'login.html', {'form': form})

@login_required
def user_logout(request):
    messages.success(request, 'You have been logged out.')
    return redirect('logout')

def movie_list(request):
    movies = Movie.objects.all()
    categories = Movie.CATEGORY_CHOICES
    search_query = request.GET.get('search', '')
    category = request.GET.get('category', '')

    if search_query:
        movies = movies.filter(title__icontains=search_query)
    if category:
        movies = movies.filter(category=category)

    return render(request, 'movie_list.html', {
        'movies': movies,
        'categories': categories,
        'search_query': search_query,
        'selected_category': category,
    })

def movie_detail(request, movie_id):
    movie = get_object_or_404(Movie, id=movie_id)
    reviews = movie.review_set.all()
    user_review = None
    if request.user.is_authenticated:
        user_review = reviews.filter(user=request.user).first()

    if request.method == 'POST':
        if not request.user.is_authenticated:
            messages.error(request, 'Please login to submit a review.')
            return redirect('login')
        if user_review:
            messages.error(request, 'You have already reviewed this movie.')
            return redirect('movie_detail', movie_id=movie.id)
        form = ReviewForm(request.POST)
        if form.is_valid():
            review = form.save(commit=False)
            review.movie = movie
            review.user = request.user
            review.save()
            messages.success(request, 'Review submitted successfully!')
            return redirect('movie_detail', movie_id=movie.id)
        else:
            form = ReviewForm()

    return render(request, 'movie_detail.html', {
        'movie': movie,
        'reviews': reviews,
        'form': form,
    })

```

```
'user_review': user_review,
})
```

Explanation of movie_detail view:

- Fetches the movie using `get_object_or_404`.
- Gets all reviews for the movie.
- Checks if the user has already reviewed the movie.
- On POST:
 - Redirects to login if user is not authenticated.
 - Prevents duplicate reviews.
 - Saves the review with the movie and user if the form is valid.
- Passes movie, reviews, form, and user's review (if any) to the template.

1. Create template in `templates/movie_detail.html`

Displays movie details, reviews, and a review form (if logged in and hasn't reviewed).

```
{% extends 'base.html' %}
{% block content %}
<h2>{{ movie.title }}</h2>

<p><strong>Description:</strong> {{ movie.description }}</p>
<p><strong>Release Date:</strong> {{ movie.release_date }}</p>
<p><strong>Category:</strong> {{ movie.category }}</p>
<p><strong>Average Rating:</strong> {{ movie.average_rating|floatformat:1 }}</p>

<h3>Reviews</h3>
{% for review in reviews %}
  <div>
    <p><strong>{{ review.user.username }}</strong> ({{ review.rating }}/5)</p>
    <p>{{ review.content }}</p>
    <p><em>{{ review.created_at }}</em></p>
  </div>
{% empty %}
  <p>No reviews yet.</p>
{% endfor %}

{% if user.is_authenticated %}
  {% if user_review %}
    <p>You have already submitted a review for this movie.</p>
  {% else %}
    <h3>Add Your Review</h3>
    <form method="post">
      {% csrf_token %}
      {{ form.as_p }}
      <button type="submit">Submit Review</button>
    </form>
  {% endif %}
{% else %}
  <p><a href="{% url 'login' %}">Login</a> to submit a review.</p>
{% endif %}
{% endblock %}
```

Explanation of movie_detail.html:

- Shows movie details (title, image, description, release date, category, average rating).
- Lists all reviews with username, rating, content, and timestamp.
- For authenticated users:

- If they've reviewed, shows a message.
- Otherwise, shows a review form.
- For anonymous users, shows a login link.

1. Update `movies/urls.py`

Add URL for movie detail.

```
from django.urls import path
from . import views

urlpatterns = [
    path('signup/', views.signup, name='signup'),
    path('login/', views.user_login, name='login'),
    path('', views.movie_list, name='movie_list'),
    path('movie/<int:movie_id>/', views.movie_detail, name='movie_detail'),
]
```

Step 7: Implement Admin Controls (Add/Edit/Delete Movies)

Explanation:

Only admins can add, edit, or delete movies. We'll create forms, views, and templates for these actions, and use Django's `user_passes_test` to restrict access to superusers.

Code:

1. Update `movies/forms.py`

Add a form for creating/editing movies.

```
from django import forms
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User
from .models import Review, Movie

class SignUpForm(UserCreationForm):
    email = forms.EmailField(required=True)

    class Meta:
        model = User
        fields = ['username', 'email', 'password1', 'password2']

class ReviewForm(forms.ModelForm):
    class Meta:
        model = Review
        fields = ['content', 'rating']

class MovieForm(forms.ModelForm):
    class Meta:
        model = Movie
        fields = ['title', 'description', 'image', 'release_date', 'category']
```

Explanation: `MovieForm` is a `ModelForm` for the `Movie` model, including all fields except `created_at`.

1. Update `movies/views.py`

Add views for adding, editing, and deleting movies, restricted to admins.

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth import login, authenticate
from django.contrib.auth.forms import AuthenticationForm
from django.contrib.auth.decorators import login_required, user_passes_test
from django.contrib import messages
```



```

from .forms import SignUpForm, ReviewForm, MovieForm
from .models import Movie, Review


def is_admin(user):
    return user.is_superuser


def signup(request):
    if request.method == 'POST':
        form = SignUpForm(request.POST)
        if form.is_valid():
            form.save()
            username = form.cleaned_data.get('username')
            messages.success(request, f'Account created for {username}!')
            return redirect('login')
    else:
        form = SignUpForm()
    return render(request, 'signup.html', {'form': form})


def user_login(request):
    if request.method == 'POST':
        form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            username = form.cleaned_data.get('username')
            password = form.cleaned_data.get('password')
            user = authenticate(username=username, password=password)
            if user is not None:
                login(request, user)
                messages.success(request, f'Welcome back, {username}!')
                return redirect('movie_list')
    else:
        form = AuthenticationForm()
    return render(request, 'login.html', {'form': form})


@login_required
def user_logout(request):
    messages.success(request, 'You have been logged out.')
    return redirect('logout')


def movie_list(request):
    movies = Movie.objects.all()
    categories = Movie.CATEGORY_CHOICES
    search_query = request.GET.get('search', '')
    category = request.GET.get('category', '')

    if search_query:
        movies = movies.filter(title__icontains=search_query)
    if category:
        movies = movies.filter(category=category)

    return render(request, 'movie_list.html', {
        'movies': movies,
        'categories': categories,
        'search_query': search_query,
        'selected_category': category,
    })


def movie_detail(request, movie_id):
    movie = get_object_or_404(Movie, id=movie_id)
    reviews = movie.review_set.all()
    user_review = None

```



```

if request.user.is_authenticated:
    user_review = reviews.filter(user=request.user).first()

if request.method == 'POST':
    if not request.user.is_authenticated:
        messages.error(request, 'Please login to submit a review.')
        return redirect('login')
    if user_review:
        messages.error(request, 'You have already reviewed this movie.')
        return redirect('movie_detail', movie_id=movie.id)
    form = ReviewForm(request.POST)
    if form.is_valid():
        review = form.save(commit=False)
        review.movie = movie
        review.user = request.user
        review.save()
        messages.success(request, 'Review submitted successfully!')
        return redirect('movie_detail', movie_id=movie.id)
    else:
        form = ReviewForm()

return render(request, 'movie_detail.html', {
    'movie': movie,
    'reviews': reviews,
    'form': form,
    'user_review': user_review,
})

@user_passes_test(is_admin)
@login_required
def movie_add(request):
    if request.method == 'POST':
        form = MovieForm(request.POST)
        if form.is_valid():
            form.save()
            messages.success(request, 'Movie added successfully!')
            return redirect('movie_list')
    else:
        form = MovieForm()
    return render(request, 'movie_form.html', {'form': form, 'title': 'Add Movie'})

@user_passes_test(is_admin)
@login_required
def movie_edit(request, movie_id):
    movie = get_object_or_404(Movie, id=movie_id)
    if request.method == 'POST':
        form = MovieForm(request.POST, instance=movie)
        if form.is_valid():
            form.save()
            messages.success(request, 'Movie updated successfully!')
            return redirect('movie_list')
    else:
        form = MovieForm(instance=movie)
    return render(request, 'movie_form.html', {'form': form, 'title': 'Edit Movie'})

@user_passes_test(is_admin)
@login_required
def movie_delete(request, movie_id):
    movie = get_object_or_404(Movie, id=movie_id)
    if request.method == 'POST':

```

```

movie.delete()
messages.success(request, 'Movie deleted successfully!')
return redirect('movie_list')
return render(request, 'movie_delete.html', {'movie': movie})

```

Explanation of admin views:

- `is_admin` : Helper function to check if a user is a superuser.
- `movie_add` : Allows admins to add a new movie. Saves the form and redirects to the movie list.
- `movie_edit` : Allows admins to edit an existing movie. Uses the same form with `instance=movie` .
- `movie_delete` : Confirms deletion with a POST request to prevent accidental deletes.
- All views use `@user_passes_test(is_admin)` to restrict access to superusers.

1. Create templates for add/edit and delete

- **Movie form template in** `templates/movie_form.html` :

```

{% extends 'base.html' %}
{% block content %}
<h2>{{ title }}</h2>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Save</button>
</form>
<a href="{% url 'movie_list' %}">Cancel</a>
{% endblock %}

```

- **Delete confirmation template in** `templates/movie_delete.html` :

```

{% extends 'base.html' %}
{% block content %}
<h2>Delete {{ movie.title }}</h2>
<p>Are you sure you want to delete this movie?</p>
<form method="post">
    {% csrf_token %}
    <button type="submit">Confirm Delete</button>
</form>
<a href="{% url 'movie_list' %}">Cancel</a>
{% endblock %}

```

Explanation of templates:

- `movie_form.html` : Used for both add and edit, with a dynamic title. Displays the form and a cancel link.
- `movie_delete.html` : Confirms deletion with a form and cancel link.

1. Update `movies/urls.py`

Add URLs for admin controls.

```

from django.urls import path
from . import views

urlpatterns = [
    path('signup/', views.signup, name='signup'),
    path('login/', views.user_login, name='login'),
    path('', views.movie_list, name='movie_list'),
    path('movie/<int:movie_id>/', views.movie_detail, name='movie_detail'),
    path('movie/add/', views.movie_add, name='movie_add'),
    path('movie/<int:movie_id>/edit/', views.movie_edit, name='movie_edit'),

```

```
path('movie/<int:movie_id>/delete/', views.movie_delete, name='movie_delete'),
]
```

1. **Update `movie_list.html` to show admin controls**

Add links for add/edit/delete for admins only.

```
{% extends 'base.html' %}
{% block content %}
<h2>Movies</h2>
{% if user.is_superuser %}
  <p><a href="{% url 'movie_add' %}">Add New Movie</a></p>
{% endif %}
<form method="get">
  <input type="text" name="search" value="{{ search_query }}" placeholder="Search by title">
  <select name="category">
    <option value="">All Categories</option>
    {% for cat in categories %}
      <option value="{{ cat.0 }}" {% if selected_category == cat.0 %}selected{% endif %}>{{ cat.1 }}</option>
    {% endfor %}
  </select>
  <button type="submit">Search</button>
</form>
<div>
  {% for movie in movies %}
    <div>
      <h3><a href="{% url 'movie_detail' movie.id %}">{{ movie.title }}</a></h3>
      
      <p>Rating: {{ movie.average_rating|floatformat:1 }}</p>
      {% if user.is_superuser %}
        <p>
          <a href="{% url 'movie_edit' movie.id %}">Edit</a> |
          <a href="{% url 'movie_delete' movie.id %}">Delete</a>
        </p>
      {% endif %}
    </div>
  {% empty %}
    <p>No movies found.</p>
  {% endfor %}
</div>
{% endblock %}
```

Explanation: Adds “Add New Movie” link and Edit/Delete links for each movie, visible only to superusers.

Step 8: Final Setup and Testing

Explanation:

The project is now complete. We'll summarize the steps to run the project and test it.

Terminal Commands:

```
# Ensure migrations are applied
python manage.py makemigrations
python manage.py migrate

# Run the development server
python manage.py runserver
```

Testing Instructions:

1. **Access the site:**

- Open `http://127.0.0.1:8000/` to see the movie list.
- Use the search box and category filter to test filtering.

2. Admin panel:

- Go to `http://127.0.0.1:8000/admin/` and log in with the superuser credentials.
- Add a few movies (use image URLs, e.g., from IMDb or placeholder images).

3. Authentication:

- Sign up at `/signup/`.
- Log in at `/login/`.
- Verify the navbar shows username and Logout for logged-in users.

4. Movie details:

- Click a movie title to view details and reviews.
- As a logged-in user, submit a review and verify you can't submit another.
- As an anonymous user, verify you see a login link instead of the review form.

5. Admin controls:

- Log in as a superuser and test adding, editing, and deleting movies.
- Verify normal users can't access these pages.

Example Movie Data (to add via admin panel):

- Title: "Sample Movie", Description: "A great movie", Image: "<https://via.placeholder.com/150>", Release Date: 2023-01-01, Category: Action.

Project Structure:

```

movie_website/
├── db.sqlite3
├── manage.py
├── movies/
│   ├── __init__.py
│   ├── admin.py
│   ├── forms.py
│   ├── models.py
│   ├── urls.py
│   └── views.py
├── templates/
│   ├── base.html
│   ├── login.html
│   ├── signup.html
│   ├── movie_list.html
│   ├── movie_detail.html
│   ├── movie_form.html
│   └── movie_delete.html
├── movie_website/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py

```

Key Django Concepts Covered:

- **Models:** Defined `Movie` and `Review` with relationships and constraints.
- **Views:** Used function-based views for all functionality.
- **Forms:** Used `ModelForm` for reviews and movies, `UserCreationForm` for signup.
- **Templates:** Created a base template with navbar and child templates with inheritance.

- **Authentication:** Leveraged Django's auth system for signup, login, logout, and permissions.
- **Admin:** Registered models for admin management.
- **Messages:** Used for user feedback.
- **URLs:** Configured project and app-level URLs.
- **Permissions:** Restricted add/edit/delete to admins using `user_passes_test`.

This project provides a solid foundation for learning Django's core concepts. You can extend it by adding more features like pagination or advanced search later.

Step: Implement Logout Functionality

Explanation:

The logout functionality will:

- Use Django's `logout` function to end the user's session.
- Display a success message using the messages framework.
- Redirect to the movie list page (as set in `LOGOUT_REDIRECT_URL`).
- Be accessible via a URL mapped to the `user_logout` view.
- Require the user to be logged in using the `@login_required` decorator.

Code:

1. Update `movies/views.py`

Modify the

`user_logout` view to use Django's `logout` function instead of redirecting to the built-in logout view.

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth import login, authenticate, logout
from django.contrib.auth.forms import AuthenticationForm
from django.contrib.auth.decorators import login_required, user_passes_test
from django.contrib import messages
from .forms import SignUpForm, ReviewForm, MovieForm
from .models import Movie, Review

def is_admin(user):
    return user.is_superuser

def signup(request):
    if request.method == 'POST':
        form = SignUpForm(request.POST)
        if form.is_valid():
            form.save()
            username = form.cleaned_data.get('username')
            messages.success(request, f'Account created for {username}!')
            return redirect('login')
    else:
        form = SignUpForm()
    return render(request, 'signup.html', {'form': form})

def user_login(request):
    if request.method == 'POST':
        form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            username = form.cleaned_data.get('username')
            password = form.cleaned_data.get('password')
            user = authenticate(username=username, password=password)
            if user is not None:
                login(request, user)
                messages.success(request, f'Welcome back, {username}!')
```

```

        return redirect('movie_list')
    else:
        form = AuthenticationForm()
    return render(request, 'login.html', {'form': form})

@login_required
def user_logout(request):
    logout(request)
    messages.success(request, 'You have been logged out.')
    return redirect('movie_list')

def movie_list(request):
    movies = Movie.objects.all()
    categories = Movie.CATEGORY_CHOICES
    search_query = request.GET.get('search', '')
    category = request.GET.get('category', '')

    if search_query:
        movies = movies.filter(title__icontains=search_query)
    if category:
        movies = movies.filter(category=category)

    return render(request, 'movie_list.html', {
        'movies': movies,
        'categories': categories,
        'search_query': search_query,
        'selected_category': category,
    })

def movie_detail(request, movie_id):
    movie = get_object_or_404(Movie, id=movie_id)
    reviews = movie.review_set.all()
    user_review = None
    if request.user.is_authenticated:
        user_review = reviews.filter(user=request.user).first()

    if request.method == 'POST':
        if not request.user.is_authenticated:
            messages.error(request, 'Please login to submit a review.')
            return redirect('login')
        if user_review:
            messages.error(request, 'You have already reviewed this movie.')
            return redirect('movie_detail', movie_id=movie.id)
        form = ReviewForm(request.POST)
        if form.is_valid():
            review = form.save(commit=False)
            review.movie = movie
            review.user = request.user
            review.save()
            messages.success(request, 'Review submitted successfully!')
            return redirect('movie_detail', movie_id=movie.id)
        else:
            form = ReviewForm()

    return render(request, 'movie_detail.html', {
        'movie': movie,
        'reviews': reviews,
        'form': form,
        'user_review': user_review,
    })

```



```

@user_passes_test(is_admin)
@login_required
def movie_add(request):
    if request.method == 'POST':
        form = MovieForm(request.POST)
        if form.is_valid():
            form.save()
            messages.success(request, 'Movie added successfully!')
            return redirect('movie_list')
    else:
        form = MovieForm()
    return render(request, 'movie_form.html', {'form': form, 'title': 'Add Movie'})

@user_passes_test(is_admin)
@login_required
def movie_edit(request, movie_id):
    movie = get_object_or_404(Movie, id=movie_id)
    if request.method == 'POST':
        form = MovieForm(request.POST, instance=movie)
        if form.is_valid():
            form.save()
            messages.success(request, 'Movie updated successfully!')
            return redirect('movie_list')
    else:
        form = MovieForm(instance=movie)
    return render(request, 'movie_form.html', {'form': form, 'title': 'Edit Movie'})

@user_passes_test(is_admin)
@login_required
def movie_delete(request, movie_id):
    movie = get_object_or_404(Movie, id=movie_id)
    if request.method == 'POST':
        movie.delete()
        messages.success(request, 'Movie deleted successfully!')
        return redirect('movie_list')
    return render(request, 'movie_delete.html', {'movie': movie})

```

Explanation of Changes in views.py:

- Imported `logout` from `django.contrib.auth` .
- Updated `user_logout` to call `logout(request)` to end the user's session.
- Kept the success message and redirect to `movie_list` , aligning with `LOGOUT_REDIRECT_URL` .
- No changes to other views, as they remain unaffected.

1. **Update** `movies/urls.py`

Map the

`user_logout` view to the `/logout/` URL, overriding the previous incorrect setup.

```

from django.urls import path
from . import views

urlpatterns = [
    path('signup/', views.signup, name='signup'),
    path('login/', views.user_login, name='login'),
    path('logout/', views.user_logout, name='logout'),
    path('', views.movie_list, name='movie_list'),
    path('movie/<int:movie_id>/', views.movie_detail, name='movie_detail'),
    path('movie/add/', views.movie_add, name='movie_add'),

```



```
path('movie/<int:movie_id>/edit/', views.movie_edit, name='movie_edit'),
path('movie/<int:movie_id>/delete/', views.movie_delete, name='movie_delete'),
]
```

Explanation of Changes in `urls.py`:

- Added `path('logout/', views.user_logout, name='logout')` to map the `user_logout` view to `/logout/`.
- Removed dependency on Django's built-in auth URLs for logout, as we're handling it directly.
- Kept all other URLs unchanged.

1. Update `movie_website/urls.py`

Remove the

`django.contrib.auth.urls` include, as we're now handling login and logout in the `movies` app. The admin URLs remain.

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('movies.urls')), # Include movies app URLs
]
```

Explanation of Changes in `movie_website/urls.py`:

- Removed `path('accounts/', include('django.contrib.auth.urls'))`, as we're fully handling authentication within the `movies` app.
- The `movies.urls` include covers all authentication URLs (signup, login, logout).

Testing the Logout Functionality

Steps to Test:

1. Run the server (if not already running):

```
python manage.py runserver
```

2. Log in:

- Go to `http://127.0.0.1:8000/login/`, enter credentials, and log in.
- Verify the navbar shows your username and a "Logout" link.

3. Log out:

- Click the "Logout" link in the navbar (points to `/logout/`).
- You should be redirected to the movie list page (`/`) and see a message: "You have been logged out."
- The navbar should now show "Signup" and "Login" links.

4. Verify restrictions:

- Try accessing `/logout/` without being logged in (e.g., via the browser URL). The `@login_required` decorator should redirect you to the login page.

Expected Behavior:

- Clicking "Logout" ends the session, shows a success message, and redirects to the movie list.
- The navbar updates to reflect the user's logged-out state.
- All other features (movie list, details, admin controls) remain unaffected.

Summary of Changes

- Fixed the `user_logout` view to use `logout(request)` and redirect to `movie_list`.
- Updated `movies/urls.py` to map `/logout/` to `user_logout`.
- Removed `django.contrib.auth.urls` from `movie_website/urls.py` to avoid conflicts.

- Reused the existing `base.html` logout link, which correctly uses `{% url 'logout' %}`.
- Ensured the `LOGOUT_REDIRECT_URL` in `settings.py` aligns with the redirect to `movie_list`.

The logout functionality is now fully implemented and integrates with the existing project. All other features (navbar, movie list, details, reviews, admin controls) remain unchanged and functional. If you need further clarification or want to add more features, let me know!